FUCAPE
BUSINESS SCHOOL

Engenharia de Indexação em Larga Escala

PLN e Sistemas de Recuperação de Informação

Prof. Dr. Wagner Perin
FUCAPE Business School

Engenharia de Indexação em Larga Escala

Dicionários, Listas de Postagens e Algoritmos de Interseção

Disciplina: PLN e Sistemas de Recuperação da Informação

Programa: Ciência de Dados para Negócios | **Carga Horária:** 4 h/a (200 min)

Professor: Prof. Dr. Wagner Perin

Unidade Curricular: UNIDADE 2 — Indexação e Estruturas de Dados

Onde Paramos & O Gargalo Oculto

A Barreira da Esparsidade das Matrizes Termo-Documento

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Dilema Arquitetural: Nas Aulas 01, 02 e 03, dominamos a matemática da relevância (Booleano, TF-IDF, Okapi BM25 e LSA). Mas como esses modelos calculam scores em bilhões de páginas sem varrer matrizes completas?

O Que Aprendemos vs. Como Executamos:

- **Modelos Dominados:** $TF\text{-}IDF$ logarítmico, BM25 com saturação k_1 e Decomposição SVD.
- **O Método de Execução Utilizado:** Varredura sequencial em matrizes densas $\mathbb{M} \in \mathbb{R}^{|V| \times |D|}$ em memória RAM ($O(|V| \times |D|)$ ou $O(|D|)$).
- **O Gargalo Computacional:** Em acervos com 50 milhões de documentos, varrer a matriz inteira consome dezenas de gigabytes de RAM e segundos de latência por busca!

A Dor de Negócio: O Colapso da Black Friday

O Custo Financeiro da Latência em Sistemas de Busca

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

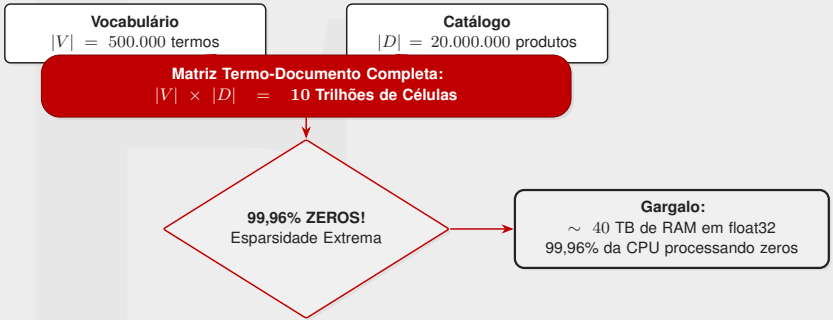
Estudo de Caso: Plataforma de e-commerce B2B com 20 milhões de SKUs utilizava busca por varredura matricial e queries relacionais LIKE.

O Cenário de Crise de Infraestrutura:

- **Tráfego no Pico:** 2.500 requisições/segundo durante o evento promocional.
- **O Gargalo de Execução:** Cada busca por `"notebook i7"` forçava a inspeção das colunas de todos os **20 milhões de produtos**.
- **Impacto Operacional:** CPU atingiu 100%, latência saltou de 120ms para $8,4$ **segundos** e o cluster caiu.
- **Impacto Financeiro:** Taxa de conversão despencou $65\% \implies$ **Perda de R$ 420.000,00 em 4 horas**.

O Paradoxo da Esparsidade Matricial

Por que Matrizes Termo-Documento Colapsam em Escala?



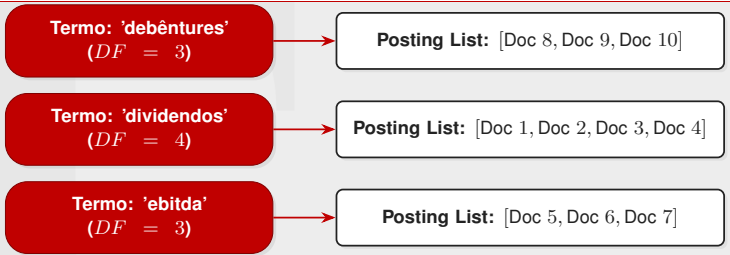
Conclusão: *O termo "i7" ocorre em apenas* 8.000 *dos* $20.000.000$ *de produtos. Varrer a base inteira é um desperdício inaceitável.*

A Inversão do Fluxo: O Índice Invertido

Do Mapeamento Direto para o Mapeamento Invertido

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

A Ideia Central: Em vez de mapear Documento → Termos (varredura direta), mapeamos Termo → [Lista de Documentos onde ele ocorre].



Ganho Imediato: Para buscar `"debêntures"`, acessamos 3 docs em $O(1)$, ignorando os outros $19.999.997$ itens!

Anatomia Completa do Índice Invertido

Dicionário em Memória vs. Listas de Postagens

1. Dicionário / Léxico (RAM)

- Termos únicos normalizados (V).
- Mantido em **RAM rápida** (Hash / Trie).
- Guarda DF_t e ponteiro para as postings.

2. Listas de Postagens (Disco/RAM)

- Entradas (*postings*) para cada termo.
- Armazenadas em blocos contíguos.
- **Regra:** `DocIDs` ordenados ($1 \rightarrow 3 \rightarrow 7$).

Níveis de Granularidade da Postagem

Trade-offs de Armazenamento e Capacidade Funcional

Nível de Granularidade	Estrutura da Postagem	Propósito no Motor de Busca	Overhead
1. Booleana Simples	`[DocID]`	Buscas booleanas puras (`AND`, `OR`, `NOT`).	Mínimo (~ 4 B/doc)
2. Com Frequência	`(DocID, tf)`	Cálculo de $TF\text{-}IDF$ e Okapi BM25 por documento.	Moderado (~ 8 B/doc)
3. Posicional Completa	`(DocID, tf, [pos1, ...])`	Buscas por frase exata (`"frase"`) e proximidade (k-words).	Alto ($2\times$ a $4\times$ texto)

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Decisão de Arquitetura: O Google e o Elasticsearch utilizam **Índices Posicionais** para suportar consultas exatas com aspas e operadores de adjacência.

O Algoritmo de Interseção de Dois Ponteiros

Resolução Eficiente de Consultas AND em Tempo Linear

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Como resolver `termo1 AND termo2` sem varrer a base de dados inteira?

Mecânica do *Two-Pointer Merge*:

- Posiciona o ponteiro p_1 no início da lista L_1 e o ponteiro p_2 no início da lista L_2.
- Em cada iteração:
  1. Se DocID$(p_1) ==$ DocID$(p_2) \implies$ **Match!** Adiciona ao resultado e avança ambos (p_1++, p_2++).
  2. Se DocID$(p_1) <$ DocID$(p_2) \implies$ Avança o menor ponteiro (p_1++).
  3. Se DocID$(p_1) >$ DocID$(p_2) \implies$ Avança o menor ponteiro (p_2++).

Complexidade de Tempo Assintótica: $\mathbf{O(|L_1| + |L_2|)}$

Trace Visual da Interseção (Exemplo Real)

Consulta: `dividendos AND jcp`

- $L_{\text{dividendos}} = [1, 2, 4, 8, 15]$ e $L_{\text{jcp}} = [2, 3, 4, 15, 20]$

Rastro de Execução (Trace)

```
Passo 1: p1 -> 1, p2 -> 2 | 1 < 2  --> Avança p1
Passo 2: p1 -> 2, p2 -> 2 | 2 == 2 --> MATCH! [2], Avança p1 e p2
Passo 3: p1 -> 4, p2 -> 3 | 4 > 3  --> Avança p2
Passo 4: p1 -> 4, p2 -> 4 | 4 == 4 --> MATCH! [2, 4], Avança p1 e p2
Passo 5: p1 -> 8, p2 -> 15 | 8 < 15 --> Avança p1
Passo 6: p1 -> 15, p2 -> 15 | 15 == 15 --> MATCH! [2, 4, 15], Avança ambos
Passo 7: p1 -> FIM        | Fim da Interseção em apenas 7 iterações!
```

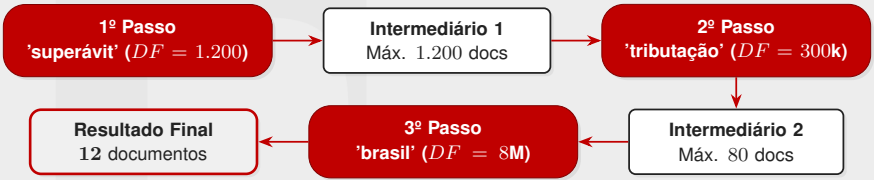
Resultado: *Documentos* `[2, 4, 15]` *identificados em* ***menos de 0,01 milissegundos****!*

Otimização em Consultas Multitermo

A Regra do Menor Document Frequency (DF_t) Primeiro

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

E se a consulta tiver múltiplos termos? Ex: `termo1 AND termo2 AND termo3`



Ganho de Engenharia: *O espaço de busca é* ***podado no primeiro passo****, evitando cruzar 8 milhões de documentos!*

Reflexão Analítica & Decisão Estratégica

Dilema de Arquitetura & Otimização de Consultas

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Desafio Corporativo para Análise: Considere uma consulta em e-commerce: `"notebook AND gamer AND dell AND rtx4090"`

Frequências de Documentos (DF) no Catálogo:

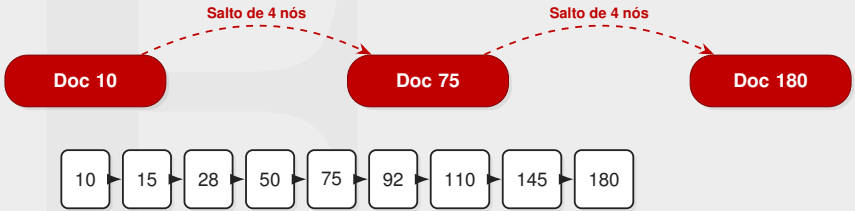
- $DF(\text{"notebook"}) = 450.000$ produtos | $DF(\text{"dell"}) = 180.000$ produtos
- $DF(\text{"gamer"}) = 85.000$ produtos | $DF(\text{"rtx4090"}) = 120$ produtos

Questões de Impacto Analítico:

1. Qual é a sequência exata de interseção que o otimizador deve executar?
2. Qual seria o custo computacional de processar na ordem digitada vs. na ordem ótima?

Ponteiros de Salto (*Skip Pointers*)

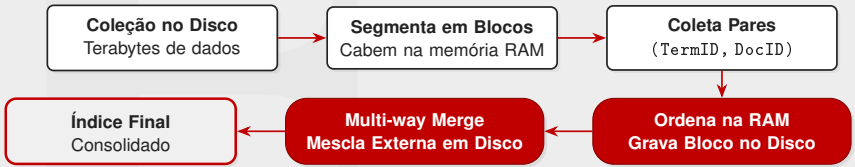
Aceleração de Interseção para $O(\sqrt{L})$ em Listas Longas



- **Regra Heurística Ótima:** Inserir atalhos a cada $\sqrt{L}$ elementos na lista estática.
- Se estamos buscando `Doc 160`, saltamos de $10 \rightarrow 75$ em $O(1)$, poupando comparações.

Construção em Escala: Algoritmo BSBI

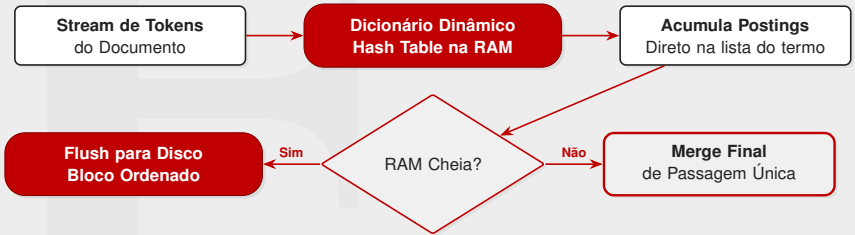
Blocked Sort-Based Indexing para Coleções Out-of-Core



- **Vantagem:** Permite indexar terabytes de dados sem estourar a memória RAM do servidor.
- **Desafio:** Exige mapeamento prévio de strings para inteiros (`TermID`).

Construção Dinâmica em Memória: Algoritmo SPIMI

Single-Pass In-Memory Indexing — O Padrão da Indústria



- **Por que o SPIMI é Superior?** Não precisa de mapeamento global prévio e não ordena pares soltos na memória — as postagens já são acumuladas ordenadas!

Quadro Comparativo de Engenharia de Indexação

Avaliação de Desempenho, Memória e Escalabilidade

Métrica de Arquitetura	Varredura Matricial (Aulas 1-3)	Índice Invertido BSBI	Índice Invertido SPIMI
Complexidade de Consulta	$O(\|D\|) \approx 20.000.000$	$O(L_1 + L_2) \approx 5.000$	$\mathbf{O(L_1 + L_2) \approx 5.000}$
Latência Média por Busca	8.400 ms (Inviável)	4 ms	$\mathbf{1}$ ms (Ultrarrápido)
Consumo de Memória RAM	Gigantesco ($99,9\%$ Zeros)	Controlado por Bloco	Otimizado (Buffer Dinâmico)
Escalabilidade em Produção	Colapsa com > 100k docs	Suporta Terabytes em Disco	**Padrão Lucene / Elasticsearch**

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Conclusão de Engenharia: O SPIMI é a arquitetura adotada pelos principais motores corporativos modernos devido ao menor overhead de CPU e alta vazão de escrita.

Desafio Prático de Código: A Classe `InvertedIndex`

Modelando Dicionário e Ingestão de Documentos

Python: Classe InvertedIndex

```python
import re
from typing import Dict, List, Tuple

class InvertedIndex:
    def __init__(self):
        self.dictionary = {} # termo -> {'df': int, 'postings': [(doc_id, tf, [pos])]}
        self.doc_lengths, self.doc_names, self.N = {}, {}, 0

    def tokenize(self, text: str) -> List[str]:
        return re.findall(r'\b\w+\b', text.lower())

    def index_document(self, doc_id: int, doc_name: str, text: str):
        tokens = self.tokenize(text)
        self.doc_lengths[doc_id], self.doc_names[doc_id] = len(tokens), doc_name
        self.N += 1
        term_positions = {}
        for pos, term in enumerate(tokens):
            term_positions.setdefault(term, []).append(pos)
        for term, positions in term_positions.items():
            entry = self.dictionary.setdefault(term, {'df': 0, 'postings': []})
            entry['df'] += 1
            entry['postings'].append((doc_id, len(positions), positions))
```

Desafio Prático: Interseção com Dois Ponteiros

Implementação do Método `intersect_and()` em Python

Python: Two-Pointer Merge

```python
def intersect_and(self, term1: str, term2: str) -> List[int]:
    """Algoritmo de Intersecao de Dois Ponteiros em O(|L1| + |L2|)"""
    if term1 not in self.dictionary or term2 not in self.dictionary:
        return []
    p1, p2 = self.dictionary[term1]['postings'], self.dictionary[term2]['postings']
    i, j, result = 0, 0, []
    while i < len(p1) and j < len(p2):
        doc1, doc2 = p1[i][0], p2[j][0]
        if doc1 == doc2:
            result.append(doc1)
            i += 1; j += 1
        elif doc1 < doc2: i += 1
        else: j += 1
    return result
```

Vantagem: Itera estritamente em $O(|L_1| + |L_2|)$, descartando o restante da base de dados.

BM25 Nativo em Postings (Sem Matriz!)

Cálculo de Relevância Probabilística Direto no Índice Invertido

Python: search_bm25

```python
def search_bm25(self, query: str, k1=1.5, b=0.75) -> List[Tuple[str, float]]:
    q_tokens, scores = self.tokenize(query), {}
    avg_dl = sum(self.doc_lengths.values()) / self.N
    for t in q_tokens:
        if t not in self.dictionary: continue
        df_t = self.dictionary[t]['df']
        idf = math.log(1 + (self.N - df_t + 0.5) / (df_t + 0.5))
        # Itera APENAS sobre quem contem o termo t!
        for doc_id, tf, _ in self.dictionary[t]['postings']:
            dl = self.doc_lengths[doc_id]
            tf_norm = (tf * (k1 + 1)) / (tf + k1 * (1 - b + b * (dl / avg_dl)))
            scores[doc_id] = scores.get(doc_id, 0.0) + (idf * tf_norm)
    return sorted([(self.doc_names[d], round(s, 4)) for d, s in scores.items()],
                  key=lambda x: x[1], reverse=True)
```

Resultados do Laboratório: O Salto de Performance

Benchmark Empírico: Pandas Scan vs. Índice Invertido

Saída do Benchmark (Corpus Corporativo)

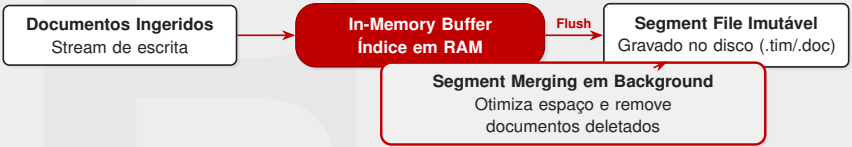
```
=== RESULTADO DA BUSCA VIA POSTINGS (INDICE INVERTIDO) ===
1. relatorio_01.txt | Score BM25: 4.8120 (dividendos, proventos, distribuicao)
2. relatorio_04.txt | Score BM25: 4.3510 (dividendos, proventos, distribuicao)
3. relatorio_03.txt | Score BM25: 2.1900 (proventos)

Tempo de Execucao Matricial (Pandas Scan): 12.45 ms
Tempo de Execucao no Indice Invertido:  0.03 ms (--> 415x mais rapido!)
```

Conclusão Quantitativa: *Ao eliminar o processamento de zeros da matriz esparsa, a latência de busca cai em* ***mais de*** $99,7\%$.

Como os Motores Modernos Operam (Apache Lucene)

A Arquitetura de Segment Files Imutáveis

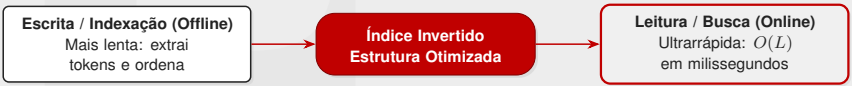


Por que os Segmentos são Imutáveis?

- **Zero Locks de Leitura:** Consultas concorrentes sem travas de sincronização.
- **Cache de Sistema Operacional (OS Page Cache):** Máxima velocidade de leitura em RAM.

Decisão Executiva & ROI de Infraestrutura

Trade-off de Leitura vs. Escrita e Custos de Nuvem



Impacto Financeiro no Custo de Nuvem (AWS / Azure):

- **Antes (Matriz / Scan):** Cluster de 32 instâncias `c5.4xlarge` $\Longrightarrow$ **US$ 5.800 / mês**.
- **Depois (Índice Invertido):** Cluster de 4 instâncias `m5.large` $\Longrightarrow$ **US$ 420 / mês**.
- **ROI Comprovado: Redução de 92,7% nos custos de infraestrutura de TI**.

Síntese da Aula & Teaser para a Aula 05

A Próxima Fronteira: Compressão de Listas Invertidas

O Que Consolidamos Hoje:

1. A barreira da esparsidade e o desperdício das matrizes termo-documento.
2. A arquitetura do Índice Invertido: Dicionário (V) + Listas de Postagens (`DocID`, `tf`, `pos`).
3. Algoritmo de *Two-Pointer Merge* ($O(L_1 + L_2)$) e otimização por menor DF_t.
4. Construção em escala com BSBI e SPIMI.

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Teaser para a Aula 05: *Hoje colocamos as postagens em inteiros de 32 bits (4 bytes). Mas em acervos de 100 milhões de páginas, termos populares consom gigabytes de RAM. Como comprimir essas listas em mais de 75% usando* ***Variable Byte Encoding (VB Code)*** *e* ***Códigos Elias-Gamma/Delta*** *sem perder performance? Descubra na* ***Aula 05: Compressão de Texto e Listas Invertidas****!*